

Python Full-Stack Development — Syllabus

Comprehensive Curriculum for End-to-End Python Development — Logic Building, SQL, APIs, React Frontend & Real-World Projects

PHASE 1: Computers, Programming & Python Foundations (Day 1–18)

Day 1–2: Introduction to Computers

- What is a computer?
- Basic components of a computer (CPU, RAM, storage, etc.)
- How computers process data
- Basic input/output devices
- 8086 Architecture, Fetch-Decode-Execute Cycle

Day 3–4: Understanding Operating Systems

- What is an Operating System (OS)?
- Functions of an OS
- Major OS components (kernel, shell, file system, etc.)
- Different ways to instruct Kernel using UI, Shell, Programming language

Day 5: Introduction to Programming Languages

- What is a programming language?
- Difference between high-level and low-level languages
- Overview of popular programming languages

Day 6: Compilers and Interpreters

- What is a compiler?
- What is an interpreter?
- Differences between compilers and interpreters
- How Python works as an interpreted language

Day 7: Setting Up Python Environment & First Script

- Why Python? Use cases & ecosystem
- Installing Python
- Setting up IDE (VS Code / Jupyter)
- Writing & running first Python script

Day 8: Basic Python Syntax & Control Flow (Intro)

- Identifiers, variables, reserved keywords and data types
 - Type casting
 - Basic operators (arithmetic, comparison, logical)
 - Simple if conditions & basic loops
-

PHASE 2: Python Basics – Strings, Collections, Functions & Modules (Day 9–18)

Day 9–10: String Manipulation

- Basic string operations
- String methods
- Indexing and slicing
- Formatting strings (f-strings)

Day 11–14: Data Structures in Python

- Lists and their methods
- Tuples and immutability
- Dictionaries: key-value pairs, methods, nested dicts
- Sets and set operations
- Methods and their use cases
- Assignments covering all data structures

Day 15–16: Functions & Modules (Part 1)

- Defining and calling functions
- Function parameters and return values
- Scope of variables (local vs global)

- Basic idea of higher-order functions
- Importing modules, using standard library

Day 17–18: Project 1 — Hotel Management System (Console)

- Project overview: Build a hotel management system to handle room bookings, customer details, and reservation management.
 - Break down into smaller tasks using control flow, loops, and functions.
 - Implement menus, booking logic, and simple in-memory data structures (lists/dicts).
 - Thoroughly test and debug the solution.
-

PHASE 3: OOP & Intermediate Python (Day 19–27)

Day 19–22: Object-Oriented Programming (OOP) Concepts (Part 1)

- Classes and objects
- Instance variables, instance methods
- Class variables (basic idea)
- Inner classes with real examples (College, Department, Student)
- Inheritance basics (is-a relationship)

Day 23–24: Project — Hospital Management System (Console)

- Models real-world hospital components: doctors, patients, nurses, pharmacy, billing.
- Demonstrates key OOP concepts: inheritance, encapsulation, polymorphism, and inner classes.
- Emphasizes is-a and has-a relationships through departments, appointments, and billing structure.

Day 25–27: OOP Advanced Concepts (Part 2)

- MRO and super() method
- Polymorphism & method overriding
- Abstract Methods (using abc module)
- Public, Private, Protected Members
- Basic design discussion: when to use inheritance vs composition

PHASE 4: Advanced Python – Decorators, Generators, Files, Regex (Day 28–40)

Day 28–30: Decorators in Python

- Understanding first-class functions & closures
- What is a decorator and why we use it
- Creating and using simple decorators
- Practical applications of decorators (logging, validation)

Day 31–32: Mini Project – Role-Based Access Control using Decorators

- Enforce access control based on user roles and permissions using decorators.
- Introduce difference between authentication and authorization using employee data.
- Use @decorator syntax to wrap business logic and dynamically allow or deny access.

Day 33–34: Advanced Decorators & Introduction to Generators

- Chaining multiple decorators
- Parameterized decorators (@decorator(arg))
- Understanding iterators & __iter__, __next__
- Introduction to generator functions and yield

Day 35–36: Generators & Mini Project – Dynamic Ticket Pricing System

- Creating custom iterators and generators
- Real-life generator use-cases (infinite sequences, streams)
- Implement a ticket sales simulator with custom iterator.
- Use a generator to dynamically calculate ticket prices based on demand.
- Demonstrate how generators optimize memory for large sequences.

Day 37–38: Logging & Exception Handling

- Importance of logging in real-world apps
- Python's logging module

- Creating and configuring loggers (file logging, log levels)
- Understanding exceptions
- Try, except, else, finally blocks
- Creating and using custom exceptions

Day 39–40: File Handling, Regular Expressions & Python Memory Management

- Reading and writing files (text, JSON)
- Working with different file modes (r, w, a, rb, wb)
- Exception handling in file operations
- Understanding regular expressions (regex)
- Using the re module in Python
- Practical regex applications: name, phone, email, PAN validation
- Overview of Python memory management: reference counting, garbage collection

PHASE 5: Major Python Project – CLI Banking System (Day 41–50)

Day 41–50: Major Project – CLI-Based Banking System

– Project Objective:

- Simulate a real-world banking environment with secure transactions, structured logic, and professional development practices.

– Real-World Project Lifecycle Training:

- Requirement gathering, flowchart creation, and logic planning before coding.
- Mimic actual software project management used in IT companies.

– Modular OOP-Based Architecture:

- Classes for users, accounts, transactions, authentication, and validation.
- Strong OOP principles: inheritance, encapsulation, has-a/is-a relationships.

– Secure Authentication:

- Optional: OTP verification using Gmail SMTP / Fast2SMS.
- Secure handling for sensitive operations (withdrawals, transfers).

– **Robust Input Validation:**

- Regex-based validation for names, phone numbers, email.

– **Console UI Enhancements:**

- Red/green color-coded CLI feedback.
- Custom formatters for clean navigation and prompts.

– **Persistent Storage:**

- Store users, accounts, and transaction records as files (e.g., daily logs).

– **Exception & Error Handling:**

- Use try-except blocks and custom exceptions to handle errors gracefully.

– **Outcomes:**

- End-to-end project: planning → coding → testing → delivery.
- Build a secure, modular, extensible banking backend in pure Python.

PHASE 6: Concurrency, SQL & FastAPI Backend (Day 51–68)

Day 51–53: Concurrency in Python

- Multithreading in Python
 - Understanding threads, GIL basics
 - Creating threads, synchronization, thread safety
- Multiprocessing in Python
 - Difference between threading and multiprocessing
 - Creating processes, basic IPC concepts
- Async Programming
 - async/await, event loop
 - Using asyncio for non-blocking I/O
 - When to use threading vs multiprocessing vs async

Day 54–56: SQL Basics & DDL/DML

- What is SQL? RDBMS & schema concepts
- DDL:
 - CREATE TABLE, ALTER TABLE, DROP TABLE
- Data types & constraints:
 - PRIMARY KEY, FOREIGN KEY (conceptual intro), NOT NULL, UNIQUE, DEFAULT, CHECK

- DML:
 - INSERT, UPDATE, DELETE
- Designing simple schemas (students, employees, customers)

Day 57–59: SQL Filtering, Aggregation & Joins

- SELECT with filtering:
 - WHERE, BETWEEN, IN, LIKE, IS NULL
- Aggregations:
 - COUNT, SUM, AVG, MAX, MIN
- GROUP BY and HAVING
- JOINS:
 - INNER JOIN, LEFT JOIN (and idea of RIGHT JOIN)
- Practical exercises with related tables (employee-department, customer-orders, etc.)

Day 60–61: FastAPI Introduction

- Why FastAPI? Compare briefly with Flask/Django/DRF
- Creating a basic FastAPI application
- Defining path & query parameters
- Pydantic models for request/response validation
- Simple request-response workflows with type-safe inputs & outputs

Day 62–63: FastAPI Dependency Injection & Error Handling

- Understanding dependency injection in FastAPI
- Creating reusable dependencies (DB session, auth stub, etc.)
- Custom exception handlers
- Middleware basics for logging/timing/security

Day 64–65: Database Integration with FastAPI

- Connecting FastAPI with PostgreSQL / MSSQL using SQLAlchemy or psycopg2
- Designing database models (ORM)
- Managing DB sessions cleanly
- Basic CRUD endpoints for a simple entity (e.g., Employee/Task or Account/Customer)

Day 66–68: Real-World API Project — Employee/Task Manager (or Bank API)

– Objective:

- Build a medium-scale backend with FastAPI + SQL DB.

– Features:

- Employee & Task creation, update, delete
- Simple authentication (login)
- Relational mappings (e.g., employee → multiple tasks)

– Practices:

- API modularization (routers)
- Input validation using Pydantic
- Error handling & proper status codes
- Basic documentation using OpenAPI/Swagger

(If pace allows, you can adapt the same concepts to expose **Bank of Python APIs** as well.)

FRONTEND & FULL-STACK APPLICATION (Day 69–120)

PHASE 7: HTML & CSS Fundamentals (Day 69–79)

Day 69–72: Web & HTML Basics

- Web vs Web Application, role of frontend vs backend
- HTML document structure (<!DOCTYPE html>, <html>, <head>, <body>)
- Headings, paragraphs, links, images
- Lists (ordered, unordered)
- Tables for structured data (rooms, patients, accounts)
- Semantic elements: header, nav, main, section, footer

Day 73–76: Forms, CSS Basics & Flexbox

- Forms: form, input, label, select, textarea, button
- HTML5 validation attributes: required, min, max, pattern
- CSS:
 - Ways to include CSS (inline, internal, external)

- Selectors (element, class, id)
- Box model: margin, padding, border, content
- Display: block, inline, inline-block
- Flexbox for layout:
- display: flex, flex-direction, justify-content, align-items, gap
- Basic responsive design with media queries

Day 77–79: Project — Hotel Management UI (Static)

- Build static multi-page UI for Hotel system:
- Home / landing page
- Rooms page (grid/cards of rooms)
- Contact/About page
- Apply semantic HTML + CSS styling + flexbox layouts
- Make layout responsive for mobile & desktop

PHASE 8: JavaScript & DOM – Applying on Hotel App (Day 80–90)

Day 80–82: JavaScript Fundamentals

- Including JS in HTML
- let, const, avoiding var
- Data types, operators
- Control flow: if/else, switch, loops (for, while)
- Arrays & objects basics
- Intro to array methods: map, filter, reduce (simple examples)

Day 83–85: DOM & Events

- DOM tree overview
- Selecting elements (getElementById, querySelector)
- Manipulating content (innerText, innerHTML, classList)
- Handling events: click, submit, input
- Form validation with JavaScript for hotel booking form

Day 86–87: Browser Storage & Intro to Fetch

- localStorage: saving and retrieving JSON
- Persisting bookings/rooms in browser storage
- Intro to fetch() (GET & simple POST) with a dummy endpoint (or simple FastAPI /ping)

Day 88–90: Project — Hotel Management App (Frontend Only)

- Extend static Hotel UI into interactive application:
 - Booking form with validation
 - Rooms dashboard rendered from JS data
 - Bookings list page (table view)
 - Filters by guest name / room type
- Store bookings in localStorage
- Load & render bookings on page load

PHASE 9: React Core & Mini Hospital App (Day 91–101)

Day 91–94: React Fundamentals

- What is React and why SPA?
- Setting up React with Vite
- JSX syntax & differences from HTML
- Functional components & props
- useState hook for local state
- Event handling in React (onClick, onChange)

Day 95–98: Lists, Forms & Component Design

- Rendering lists with .map and key prop
- Controlled components (form fields tied to state)
- Basic client-side validation in React
- Simple component architecture: splitting UI into reusable parts

Day 99–101: Project — Mini Hospital App (React, Mock Data)

- Use mock arrays for patients, doctors, departments.

Features:

- Patients list page:
 - Render patients from state
 - Search / filter by name or department
- New patient form:
 - Controlled form with validation
 - Add patient to in-memory list
- Purpose:
 - Apply core React (components, props, state, lists, forms)
 - Use familiar Hospital domain from Python projects

PHASE 10: React + API + Advanced Patterns (Day 102–108)

Day 102–104: React Data Fetching & Routing

- useEffect hook for fetching data
- Dependency array patterns (avoid infinite loops)
- Consuming FastAPI endpoints from React (basic GET/POST)
- Introduction to React Router:
 - BrowserRouter, Routes, Route, Link
 - Route params with useParams
- Apply routing to hospital-style pages (patients list / details)

Day 105–108: Context, Custom Hooks & Performance Basics

- Context API:
 - Creating AuthContext (structure)
 - Providing and consuming context via useContext
- Custom hooks:
 - useAuth for login/logout + user state
 - useApi or useFetch for API calls
- Protected routes:

- Restrict access based on auth state
 - Performance basics:
 - When React re-renders
 - React.memo for child components
 - useCallback and useMemo with real scenarios (like filtering transactions)
-

PHASE 11: Final Full-Stack Capstone — Bank of Python Portal (Day 109–120)

Day 109–120: Project — Bank of Python Full-Stack Web Portal

Backend:

- Use Bank domain & logic built earlier in CLI project.
- Extend/implement FastAPI endpoints for:
- Customers, accounts, transactions
- JWT-based login & role-based access (admin/customer)

Frontend (React):

1. Authentication & Layout

- Login page (React form) → call /login API, store token in AuthContext
- Logout and global auth state management with useAuth
- Protected routes (dashboard, accounts, transactions)

2. Customer Dashboard

- Summary cards: total balance, number of accounts, recent transactions
- Data fetched from Bank APIs

3. Accounts Module

- Accounts list page:
 - Account number, type, balance
 - Filter by account type
 - Simple pagination
- Account details page (/accounts/:id):
 - Detailed view + recent transactions

4. Transactions Module

- Transactions list with:

- Date, description, type (credit/debit), amount, running balance
- Filters: type and date range
- Pagination (client-side or server-side, as per implementation)
- Transfer Money page:
 - Controlled form for from-account, to-account, amount, description
 - Validation & error messages
 - POST /transactions/transfer + success/error toast

5. **Admin Module (Optional but Valuable)**

- Admin Customers Page:
 - List all customers
 - Create new customer
- Admin Accounts Page:
 - Create account for existing customer

6. **Architecture & Code Quality**

- Organize project into:
 - src/pages, src/components, src/hooks, src/contexts, src/services
- Use useApi for API abstraction, handling errors and loading states
- Optimize key components with React.memo, useCallback, useMemo where needed